

Worksheet — 2.1 Elements of Computational Thinking

Name: ___ Date: _____

OCR A Level Computer Science (H446) — Component 02 — Spec ref 2.1.1–2.1.5

Time: ~30 min.

Reminder: this topic is examined by **application to a scenario**, not by reciting definitions. Always name the scenario's *actual* inputs, conditions and sub-procedures.

The scenario — *BookIt* library room-booking system

Throughout this worksheet you will work on one scenario: a small program called **BookIt** that lets students book a study room in a library. A student picks a room and a time slot, and the system either confirms the booking or rejects it. It shows the rooms that are free, takes the student's ID and chosen slot, checks availability, then stores the booking and prints a confirmation.

Keep this scenario in mind for every task below.

Task 1 — Key-term match (the five thinking skills)

Draw a line (or write the matching letter) to connect each thinking skill to its definition.

Thinking skill	Definition
1. Thinking abstractly	A. Identifying decision points where the flow of control branches on a condition.
2. Thinking ahead	B. Spotting parts of a problem that can run at the same time.
3. Thinking procedurally	C. Removing/hiding unnecessary detail so only relevant information remains.
4. Thinking logically	D. Breaking a problem into an ordered sequence of steps and sub-procedures.
5. Thinking concurrently	E. Planning before coding: inputs/outputs, preconditions, caching, reusable components.

Answers: 1 ___ 2 ___ 3 ___ 4 ___ 5 ___

Task 2 — Thinking ahead: inputs, outputs and preconditions (*BookIt*)

(a) List **three inputs** the *BookIt* system needs from the user.

1. _____

2. _____

3.

(b) List **two outputs** the system produces.

1.

2.

(c) State **one precondition** that must be true *before* a booking can be confirmed.

Tip: "Validate the student ID" is a **process**, not an input or output — don't list it in (a) or (b).

Task 3 — Abstraction: keep or remove? (BookIt)

You are modelling each *room* in BookIt. For each piece of detail below, tick whether you would **KEEP** it in the model or **REMOVE** it, and write a one-line reason for the first three.

Detail about a room	KEEP	REMOVE
Room number / ID	☐	☐
Maximum capacity (seats)	☐	☐
Colour of the carpet	☐	☐
Which time slots are already booked	☐	☐
The make of chairs in the room	☐	☐

Reasons:

KEEP example — _____

REMOVE example — _____

Why is the removed detail irrelevant **to this problem** specifically? _____

Task 4 — Decomposition: break it into sub-problems (BookIt)

Decompose the "make a booking" task into **four to six** sub-problems / sub-procedures. Then number them in the order they must run.

Order	Sub-procedure
_____	_____
_____	_____
_____	_____
_____	_____
_____	_____
_____	_____

Which sub-procedure **depends on the output** of an earlier one? State which depends on which.

Task 5 — Thinking logically: decision points (BookIt)

Identify **two decision points** in BookIt. For each, write the **exact Boolean condition** and state what happens in **both** branches.

Decision point 1

Condition (Boolean): IF _____

If TRUE: _____

If FALSE: _____

Decision point 2

Condition (Boolean): IF _____

If TRUE: _____

If FALSE: _____

Task 6 — Thinking concurrently: what can run at once? (BookIt)

(a) Name **two processes** in BookIt that could run **concurrently** because they have **no data dependency** between them.

1. -----

2. -----

(b) Name **two steps that must stay sequential** (one depends on the other) and say why.

(c) State **one benefit** and **one trade-off** of running parts of BookIt concurrently.

Benefit: _____

Trade-off: _____

Task 7 — Abstraction vs decomposition: which is which?

For each statement, write **A** (abstraction) or **D** (decomposition).

1. Ignoring the carpet colour when modelling a room. ____
2. Splitting "make a booking" into check-availability, store-booking and confirm. ____
3. Representing the whole library as a list of rooms with free/busy flags. ____
4. Breaking the program into separate display, validate and save procedures. ____
5. Treating "savings" and "current" accounts as one general `Account` model. ____
6. Dividing the timetable problem into "morning slots" and "afternoon slots" handlers. ____

Challenge box

Caching in BookIt. The "rooms currently free" list is requested by many students at once and is expensive to recompute from the database every time. Explain how **caching** this list would help, and give **one trade-off**, including what could make the cached data go **stale** and how you would handle it.

Answer key

Task 1 — Key-term match 1 → C, 2 → E, 3 → D, 4 → A, 5 → B.

Task 2 — Inputs / outputs / preconditions - (a) Inputs (any three): student ID; chosen room (room number); chosen date/time slot; (optionally) duration / number of people. - (b) Outputs (any two): the list of available rooms displayed; a confirmation message / booking reference; or a rejection message if unavailable. - (c) Precondition (any one): the student ID must be valid/registered before a booking is accepted; **or** the chosen room and slot must exist; **or** the chosen slot must currently be free. (Note: validating the ID and checking availability are *processes*, not inputs/outputs.)

Task 3 — Abstraction (keep/remove) - KEEP: room number/ID, maximum capacity, which slots are already booked — these are needed to decide and store a booking. - REMOVE: carpet colour, make of chairs — irrelevant to whether a room can be booked. They are irrelevant *to this problem* because the booking decision depends only on identity, capacity and availability, never on décor.

Task 4 — Decomposition (model order) 1. Get/validate the student ID. 2. Display the list of available rooms. 3. Get the student's chosen room and time slot. 4. Check the chosen slot is still free (availability check). 5. Store the booking in the database. 6. Print/display the confirmation (or rejection). Dependency example: step 5 (store) depends on the output of step 4 (the availability check must pass first); step 4 depends on the chosen room/slot from step 3.

Task 5 — Decision points (examples) - DP1: `IF studentID is valid` → TRUE: continue to room selection; FALSE: show "unknown ID / please register" and stop. - DP2: `IF chosenSlot is free` (e.g. `IF slotStatus = "free"`) → TRUE: store the booking and confirm; FALSE: reject and ask the student

to choose another slot. - (Other valid conditions: IF seatsRequested <= roomCapacity, IF bookingTime >= openingTime AND bookingTime <= closingTime.)

Task 6 — Concurrency - (a) Independent (no data dependency): e.g. loading the available-rooms list for the screen **while** the system logs the previous request; refreshing one room's availability while another student's confirmation email is sent; rendering the UI while a background database query runs. - (b) Sequential (must stay ordered): you must **check availability before storing the booking**, and **store the booking before printing the confirmation** — each needs the previous step's result. - (c) Benefit: faster/more responsive — the interface stays usable while a slow query runs, and multiple students can be served at once on multi-core hardware. Trade-off: harder to design and debug; risk of a **race condition** if two students book the same slot at the same instant (needs synchronisation/locking), and concurrency adds overhead so it is not automatically faster.

Task 7 — Abstraction vs decomposition 1. A 2. D 3. A 4. D 5. A 6. D

Challenge box — caching Caching stores the "rooms free" list in fast memory so repeated requests are served without re-querying the database each time — this is faster, reduces database load/traffic and improves responsiveness when many students ask at once. Trade-off: it uses extra storage, and the cached list can become **stale** when a booking or cancellation changes availability but the cache still shows the old state — a student could be shown a room that has just been taken. Handle this by **invalidating/updating the cache** whenever a booking or cancellation occurs, or by setting a short expiry time so the list is refreshed regularly.